

Solution for Table Drawing

This problem asks us to reconstruct a complex grid of borders using the minimum possible number of operations. Trying to build the table from a blank sheet forward in time is difficult: you would have to keep track of which stray lines need to eventually be erased by future E commands. Instead, one possible key to solving this problem is to think in reverse.

Time Reversal

Imagine we start with the *finished* table and want to deconstruct it back to a blank sheet.

We can start by asking: “What was the very last operation we performed?” Clearly, the result of this particular operation must be fully visible: either as a fully-drawn border, or as a fully-erased area.

Suppose we somehow pick one such operation and undo it. We get a new figure, and in this figure we’ll have a new third type of edges: the edges that were affected by the last operation now become “wildcards”. We no longer care about their state because we know that the very last operation will turn all of them into their correct state.

Thus, in reverse-time, our grid will consist of three types of edges:

1. **Must-draw:** Edges that have to be drawn.
2. **Cannot-draw:** Edges that have to finish empty.
3. **Don’t-care:** Edges that can finish in either state because we have already made sure they’ll be OK.

Each time we pick the next operation to apply in reverse, it means that in the *forward* timeline, this operation is applied *as the last step* when the current figure is produced. Thus, its effect is as follows:

- Any one of the draw operations: In forward-time, this draws some lines. We can only undo such an operation if all the edges it involves are must-draw or don’t-care. As soon as there’s a cannot-draw edge among the affected edges, the draw operation is invalid — it couldn’t have been the last one that produced the current state.

Undoing this operation turns all the affected must-draw edges into don’t-cares/

- Erase operation: Essentially the same, but this time all affected edges must be of the types cannot-draw and don’t-care.

By thinking in reverse, our goal simplifies to finding a sequence of operations that turns **all** must-draw and **some** cannot-draw edges of the given figure into don’t-care edges.

A greedy strategy

We can represent the current state as two 0/1 matrices: one for the locations of must-draw edges and the other for cannot-draw edges. (Probably the most convenient representation simply reuses the locations of edges in the ASCII art drawing and has zeros everywhere else.)

Once we have the current state, we can precompute 2D prefix sums for both arrays. This allows us to make constant-time checks whether a particular operation is currently possible (i.e., undoable): suitable prefix sums can tell us whether some line or area contains some edges of the forbidden type.

We can now try solving our problem greedily: in each iteration, iterate over all* operations to see which ones can currently be undone, pick the best one among them and undo it.

(Of course, the iteration can be pruned by breaking early whenever we know that bigger operations will contain the same forbidden edge that's already present in the current one.)

"Pick the best one" is a place where we can think and experiment. A decent metric is simply the number of edges we turn into don't-cares. A somewhat better metric gives a higher weight to turning must-draw edges into don't-cares.

This greedy solver works great for the steps that still involve decently large chunks of the drawing. However, it suffers from a "long tail" problem: Towards the end of the reverse-time process, the board becomes very sparse and the greedy algorithm will be using entire operations just to draw or erase a few edges here and there.

Integer Linear Programming (ILP)

If we forget about the erase operation and just draw, the problem transforms into a simpler and better-known one: a special case of the Set Cover problem. Each operation draws a set of edges, and we want to pick the smallest number of these operations such that the union of their sets is the set of all edges that should be drawn.

We can solve this problem by formulating it as an integer linear program and then running an ILP solver and telling it to output the best solution it can find. The program will look as follows:

- We pre-generate a list of all possible operations that draw some must-draw edges and no cannot-draw edges. (We might want to do some pruning here and at least avoid generating completely equivalent operations such as different T operations with rectangles that have the same top edge and just differ in height.)
- For each such operation we will have a binary variable whether we use it.
- We set the objective to minimize the sum of all variables.
- For each must-draw edge we'll have a constraint that it must be drawn (i.e., the sum of variables that represent operations which draw our edge must be at least 1).

Combining both approaches

By combining both approaches we can get better solutions than from either of them separately. We can start with the greedy approach to quickly undo the large to medium sized operations, including ones where we erase parts of the grid, and once the quality of the individual operations drops below some threshold, we'll give up on erasing and instead we'll switch to ILP to draw the remaining must-draw edges efficiently.

Remember that the final solution then *starts* by the solution to the ILP, and only afterwards we apply, in reverse order, the operations undone by the greedy phase.